

StackRAG

A LLM and RAG powered AI Coding Assistant as a VS Code Extension

Software Project Lab 3 Final Report, 2026

Course: SE801

Submitted by

Shifat Jahan Shifa

BSSE Roll No: 1301

Exam Roll: 2322205

Institute of Information Technology

University of Dhaka

Supervised by

Dr. Kazi Muheymin-Us-Sakib

Professor

Institute of Information Technology

University of Dhaka



Institute of Information Technology

University of Dhaka

Submission Date: 26.02.2026

Letter of Transmittal

February 26, 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Final Report Submission - StackRAG: A LLM and RAG powered AI Coding Assistant as a VS Code Extension.

Dear Sir,

I am submitting the technical report for my Software Project Lab 3, *StackRAG: A LLM and RAG powered AI Coding Assistant AS VS Code Extension*. This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Shifat Jahan Shifa
BSSE-1301
Institute of Information Technology
University of Dhaka

26.02.2026

Supervisor's Signature

Acknowledgment

I would like to express my sincere gratitude to the Institute of Information Technology, University of Dhaka, for the opportunity to work on the "StackRAG: A LLM and RAG powered AI Coding Assistant AS VS Code Extension" project. This journey has been both rewarding and enlightening. A special thanks to my supervisor, **Dr. Kazi Muheymin-Us-Sakib**, Professor at the Institute of Information Technology, whose guidance and encouragement were invaluable throughout the project. His insights helped shape the project's direction and fueled my commitment to excellence. I also appreciate my classmates' camaraderie and valuable feedback, which added depth to this experience and helped refine the project.

Abstract

Modern software development frequently involves searching for solutions to programming errors, bugs, and implementation challenges. It often requires developers to manually browse large volumes of online resources such as Stack Overflow and other sites. This process is time-consuming and error-prone, especially in large projects. Traditional search-based approaches don't help developers properly to solve their problem.

To address these challenges, we propose **StackRAG**, a retrieval-augmented generation based system designed to provide context-aware programming assistance by using the Stack Overflow knowledge. StackRAG retrieves relevant question answer pairs based on the developer's query and code context, and then uses large language models to generate concise, accurate responses. By giving priority to the expert mind, this tool helps developers solve their problem with minimal facing of hallucination.

The provided tool is expected to help developers solve their problem effectively by using the knowledge of stack overflow and the knowledge of large language model itself.

Table of Contents

Letter of Transmittal.....	1
Acknowledgment.....	2
Abstract.....	3
1. Introduction.....	6
1.1 Motivation.....	6
1.2 Project Description.....	9
1.3 Project Scope.....	9
1.4 Timeline.....	10
1.5 Purpose of Document.....	10
2. Project Description.....	10
2.1 Quality Function Deployment.....	10
2.2.1 Normal Requirements.....	11
2.2.2 Expected Requirements.....	11
2.2.3 Exciting Requirements.....	11
These features go beyond user expectations, providing unique value. Their presence can greatly enhance user satisfaction.....	11
2.2 Usage Scenario.....	11
3. Scenario-Based Modeling.....	13
3.1 Use Case Diagram.....	13
3.1.1 Level 0:.....	13
3.1.1 Level 1:.....	14
3.1.1 Level 1.1:.....	16
3.1.1 Level 1.2:.....	16
3.1.1 Level 1.4:.....	17
3.1.1 Level 1.4.3:.....	18
4. Data Based Modeling.....	19
5. Class-Based Modeling.....	19
5.1 Analysis Classes.....	20
5.2 Class Cards.....	20
5.3 CRC Diagram.....	25
6. Architectural Design.....	25
6.1 Architectural Context Diagram.....	25
6.2 Archetypes.....	27
6.3 Top Level Components.....	29
6.4 Mapping Requirements to Software Architecture.....	30
7. Methodology.....	30
7.1 Workflow diagram.....	31
7.2 knowledge Construction.....	31
7.2.1 Fetch SO posts.....	31
7.2.2 Ingest SO posts.....	31
7.2.3 Create BM25 retriever.....	31
7.3 Output Generation.....	32

7.3.1 Extract keywords.....	32
7.3.2 Retrieve relevant information from kb.....	32
7.3.3 Augmentation.....	32
7.3.4 Generate solution.....	32
7.4 Codebase Indexing.....	32
7.4.1 Chunks the code files.....	32
7.4.2 Ingest the code chunks.....	32
7.4.3 Retrieve the related code chunks.....	33
8. Component Level Design.....	33
8.1 Identify design classes.....	33
8.2 Persistent data sources.....	33
8.2.1 Databases and files.....	33
8.2.2 Identify design classes of persistent data sources.....	33
8.3 Elaboration of behavioral representations of design classes.....	33
8.4 Elaboration of deployment diagram.....	38
8.5 Code implementation.....	39
SECTION 1: STACKRAG/ (VS Code Extension).....	39
SECTION 2: BACKEND/ (Python Fast API).....	42
SECTION 3: KB/ (Knowledge Base Ingestion).....	46
9. User interface design.....	47
9.1 User Analysis.....	47
9.2 Task Analysis.....	47
9.3 User Manual.....	48
10. Preliminary Test Plan.....	54
10.1 High-Level Description of Testing Goals.....	54
10.2 Test Cases.....	55
Conclusion.....	62
Reference.....	62

List of Figures

Figure 1: Screenshot of the responses of the question.....	06
Figure 2: Survey questions.....	07
Figure 3: Survey question-1	07
Figure 4: Survey question-2	08
Figure 5: Survey question-3	08
Figure 6: Survey question-4	08
Figure 7: Project Timeline.....	10
Figure 8: Level 0 (StackRAG)	13
Figure 9: Level 1 (Modules of StackRAG)	14
Figure 10: level 1.1 (knowledge base system).....	16
Figure 11: level 1.2 (codebase indexing system)	17
Figure 12: level 1.4 (answer generation system).....	17
Figure 13: level 1.4.3 (SO posts retriever system)	18
Figure 14: CRC Diagram of StackRAG.....	25

Figure 15: Architectural Context Diagram	27
Figure 16: Layered Architecture.....	27
Figure 17: Archetypes	28
Figure 18: Top Level Components.....	30
Figure 19: Deployment diagram.....	31
Figure: 22 (extension market place).....	39
Figure: 23(extension installation).....	39
Figure: 24 (installed extension in the activity bar).....	39
Figure: 25 (the landing page).....	40
Figure: 26 (a project of python and js).....	40
Figure: 27 (mode selection).....	40
Figure: 28 (enable comparison mode).....	41
Figure: 29 (enable indexing option and indexing is completed).....	41
Figure: 30 (typing the prompt).....	42

List of Tables

Table-1: Schema table.....	19
Table-1: CRC card (Models).....	20
Table-3: CRC card (KeywordExtractor).....	21
Table-4: CRC card (Retriever).....	21
Table-5: CRC card (CustomEnsembleRetriever).....	21
Table-6: CRC card (AnswerGeneration).....	22
Table-7: CRC card (SOPostsFetcher).....	22
Table-8: CRC card (Ingestion).....	22
Table-9: CRC card (BuildAndSaveBM25).....	23
Table-10: CRC card (JSChunker).....	23
Table-11: CRC card (PYChunker).....	23
Table-12: CRC card (CodeIngestion).....	24
Table-13: CRC card (CodebaseIndexManager).....	24
Table-14: CRC card (VSCodeExtensionManag.....	25
Table-15: Requirements Mapping with components.....	30
Table-16: Test Plan -1	32
Table-17: Test Plan -2	32
Table-18: Test Plan -3	33
Table-19: Test Plan -4	33
Table-20: Test Plan -5	33
Table-21: Test Plan -6	33
Table-22: Test Plan -7	34
Table-23: Test Plan -8	34
Table-24: Test Plan -9	34
Table-25: Test Plan -10	35
Table-26: Test Plan -11	35
Table-27: Test Plan -12	35

1. Introduction

Stack Overflow has long been a trusted platform for developers to find community-verified solutions. Nowadays, with the rise of Large Language Models like ChatGPT, Gemini etc, many developers also rely on them for quick answers. However, each approach has its drawbacks. Searching through Stack Overflow threads can be time-consuming and requires filtering through multiple posts to find a clear solution. On the other hand, Large Language Models can sometimes produce answers that seem correct but are actually inaccurate or unreliable.

Our proposed tool **StackRAG** searches through Stack Overflow posts related to python and javascript languages. It fetches the accepted answers of questions and builds a question answer knowledge base. In the runtime, developers ask questions. The tool searches through the knowledge base to find the question-answer pairs relevant to the query. Then it passes the retrieved information along with the codebase context to Large Language Models to generate responses.

1.1 Motivation

One day, while I was practicing different string manipulation operations, a question came to my mind. The question is “how to reverse strings in python?”. I asked chatgpt and I got the answer. But later on, a curiosity sparked in me. I opened a browser tab and searched for the same question. I got several results. I opened a result from stack overflow. To my utter surprise, I found that the expert minds here discussed a lot of approaches towards the question. Some said what is the optimal way to solve this.

Motivating example:

Q: how to reverse strings in python?

```
python

def reverse_string(s):
    return ''.join(reversed(s))

# Example
print(reverse_string("hello"))
```

LLM generated

What is the best way of implementing a reverse function for strings?

325

My own experience with this question is academic. However, if you're a pro looking for the quick answer, use a slice that steps by `-1`:

```
python

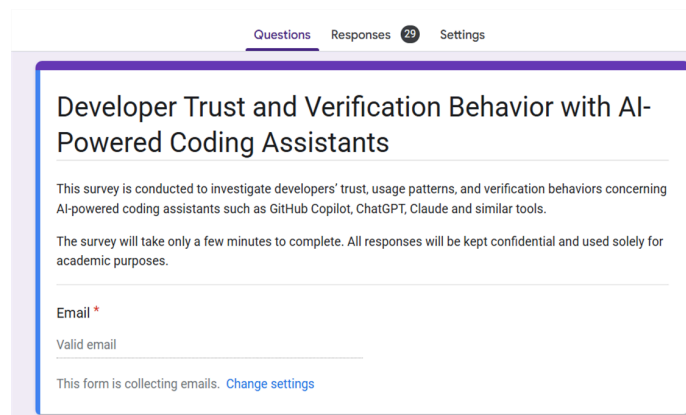
>>> 'a string'[::-1]
'gnirts a'
```

Copy

Stack Overflow discussion

Figure 1: Screenshot of the responses of the question

I initially got motivation from this. But later on, to have a stronger motivation, I conducted a survey. I collected 29 responses. **24** are **IIT** students and **5** are **software developers** from **Cefalo**.



Questions Responses 29 Settings

Developer Trust and Verification Behavior with AI-Powered Coding Assistants

This survey is conducted to investigate developers' trust, usage patterns, and verification behaviors concerning AI-powered coding assistants such as GitHub Copilot, ChatGPT, Claude and similar tools.

The survey will take only a few minutes to complete. All responses will be kept confidential and used solely for academic purposes.

Email *

Valid email

This form is collecting emails. [Change settings](#)

Figure 2: Survey questions

I asked them several questions. The questions and response results from the survey are given below:

Q1:

Do you use VS Code editor?
29 responses

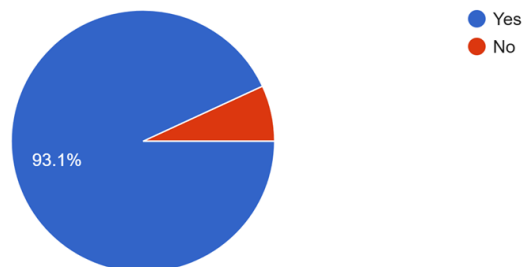


Figure 3: Survey question-1

Q2:

Do you always trust the results generated by your AI coding assistants?
29 responses

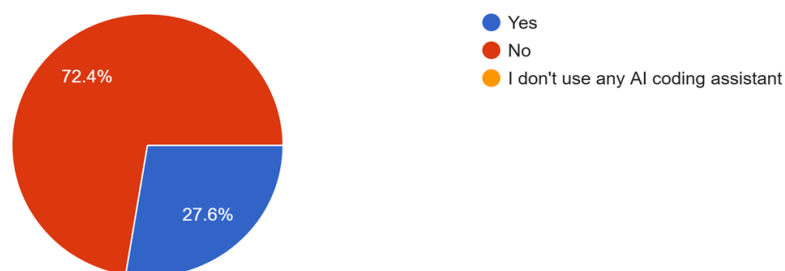


Figure 4: Survey question-2

Q3:

Do you cross-check or verify AI-generated coding solutions using resources such as Stack Overflow, Medium articles, or other online developer communities?

29 responses

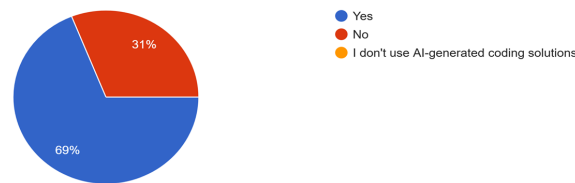


Figure 5: Survey question-3

Q4:

If your answer is yes, then select the websites you use to verify AI generated answers.

20 responses

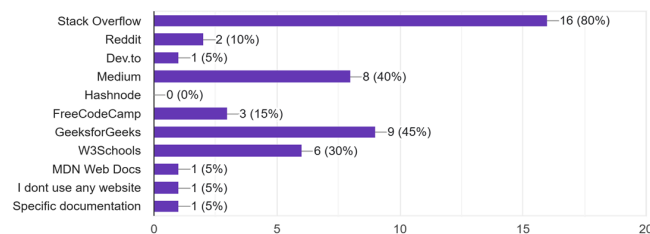


Figure 6: Survey question-4

So, we are proposing **StackRAG**, the VS Code Extension which will combine expert knowledge from Stack Overflow with Large Language Models.

1.2 Project Description

This introduces **StackRAG**, a VS Code extension to help developers solve their coding problems by using the strengths of Stack Overflow with Large Language Models. It focuses on supporting python and javascript languages. Stack Overflow has long been a trusted platform for developers to find community-verified solutions. Nowadays, with the rise of Large Language Models like ChatGPT, Gemini etc, many developers also rely on them for quick answers. However, each approach has its drawbacks. Searching through Stack Overflow threads can be time consuming. Moreover it requires filtering through multiple posts to find a clear solution. On the other hand, Large Language Models can sometimes produce answers that seem correct but are actually inaccurate or unreliable.

Our proposed tool **StackRAG** searches through Stack Overflow posts related to python and javascript languages. It fetches the accepted answers of questions and builds a question answer knowledge base. In the runtime, developers ask questions. The tool searches through the knowledge base to find the question-answer pairs relevant to the query. Then it passes the retrieved information along with the codebase context to Large Language Models to generate responses.

Broadly the features are:

1. **Interactive Chat:** Generates responses to the developer queries and attaches the links of stack overflow posts used to generate the answers.
2. **Bug fixing:** Suggests potential fixes for the developer provided coding errors.
3. **Code refactoring:** Suggests meaningful improvements to make existing code cleaner, more efficient and maintainable.
4. **Comparative analysis :** Shows the output of a developer query generated by StackRAG and VSCode Github copilot side by side in the extension UI.
5. **VS Code extension:** it will be a vscode extension.

1.3 Project Scope

The scope of **StackRAG** is mentioned below:

- It will be a VSCode extension.
- It will have general chat, bug fixing and code refactoring facilities.
- It is powered by a RAG system and LLM.
- It will provide users facility to compare the tool output with vscode github copilot interface.

1.4 Timeline

The estimated timeline for StackRAG is delineated below:

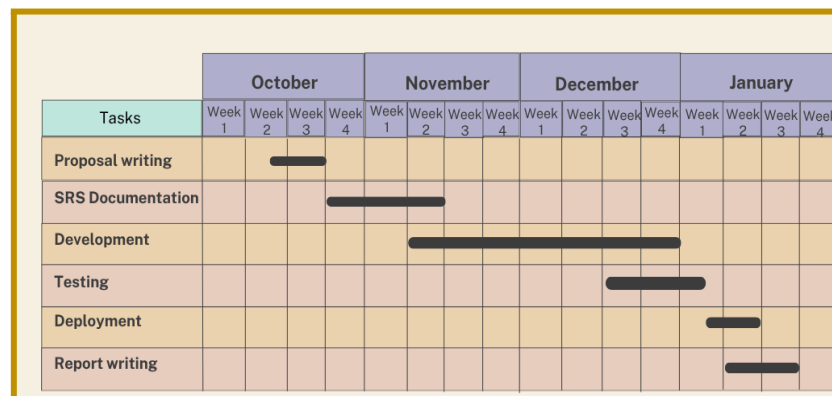


Figure 7: Project Timeline

1.5 Purpose of Document

The purposes of this document are:

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

This document outlines the Quality Function Deployment (QFD) process for StackRAG. The development of StackRAG's QFD began with an extensive review of relevant research. Their findings offered a deep understanding of the domain and informed our approach.

In addition to this research, we engaged directly with key stakeholders from the software development industry. By consulting with developers across various organizations, we gained valuable perspectives on their needs and challenges, ensuring that StackRAG addresses real-world concerns effectively.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. For this project, the following requirements are identified

2.2.1 Normal Requirements

These are fundamental features that users expect.

1. **Interactive Chat:** The tool provides interactive chat facility for developers to ask coding related questions and know the answers.
2. **Code Refactoring:** The tool provides code refactoring mode to refactor code.
3. **Bug Fixing:** The tool provides fix mode to refactor code.
4. **Integration with VSCode:** Users can access all its functionalities directly within the VSCode editor without needing to switch tools.

2.2.2 Expected Requirements

These features directly impact user satisfaction based on how well they are implemented.

1. **Context usage for generating responses:** It will use the currently open code file as a context to have consistent and accurate response generation.
2. **Appending links of related Stack Overflow Posts:** It will append the links of related Stack Overflow posts with the generated responses so that users can navigate to the links and see the posts used to generate responses.

2.2.3 Exciting Requirements

These features go beyond user expectations, providing unique value. Their presence can greatly enhance user satisfaction.

1. **Side by side view of StackRAG and VSCode github copilot response:** Providing the side by side view of the response of StackRAG and VSCode github copilot for a problem for users to compare between them.

2. **Codebase indexing:** Incorporating on demand codebase indexing by using RAG architecture to fetch relevant code context and generate accurate, context-specific responses.
3. **Codebase summary:** Providing the comprehensive summary of the codebase.

2.2 Usage Scenario

StackRAG is a VSCode extension designed to help developers solve their coding problems with the help of LLM and Stack Overflow. It provides chat facility for developers to ask coding related questions, code refactoring option, context-aware fix suggestions for coding bugs and on demand codebase indexing for getting more context-aware responses. Along with responses, StackRAG provides the related Stack Overflow posts' links. This helps developers verify the provided solution with the Stack Overflow posts and gain more insights about the topic from the posts. The usage scenario for that particular tool is delineated below:

Installation

Users can download and install the StackRAG extension from the VSCode Marketplace.

Chat Facility to ask coding related questions

StackRAG provides a chat facility for developers to ask coding related questions. It provides answers to the questions with the help of the relevant Stack Overflow posts retrieved from knowledge base and LLM. It considers code files as context if users attach code files while asking the questions.

Potential Fix Suggestions for Bugs

StackRAG provides potential bug fixing suggestions. Users can view and apply the recommended fixes. It also explains the fix suggestions along with relevant Stack Overflow posts' links helping developers understand why the error occurred and how the fix resolves it.

Potential suggestions for Code Refactoring

StackRAG provides potential code refactoring suggestions. Users can view and apply the recommended code refactoring. It also explains the code refactoring suggestions along with relevant Stack Overflow posts' links helping developers understand how the suggestions refactor the code.

Side by side view of responses from both StackRAG and VSCode GitHub Copilot

StackRAG provides an option for users to have responses for a query generated from both the extension itself and VSCode GitHub Copilot. It helps users compare between the responses and choose one which they think is more applicable to their problem.

Codebase indexing for more context-aware responses

StackRAG provides an on demand codebase indexing facility for users to have more context-aware responses. After the initial indexing of the codebase, Users can re-sync the codebase indexing anytime.

3. Scenario-Based Modeling

Scenario-based modeling is a method used in software development and design to create models that illustrate how a system behaves or responds to different real-world situations or scenarios. These scenarios are usually based on user interactions, events, or conditions, helping developers visualize and understand how the software will function in various contexts. By simulating these scenarios, developers can identify potential issues, make informed design choices, and ensure that the software meets user needs in diverse usage situations.

3.1 Use Case Diagram

A use case diagram is a visual tool in software engineering that depicts how a system interacts with external entities or actors to accomplish specific tasks or goals. It shows the connections between use cases (which represent system functions) and actors (which represent external users or systems). Use case diagrams are typically created during the requirements gathering phase, where system functionalities and user interactions are identified and then mapped out using specialized modeling tools.

3.1.1 Level 0:

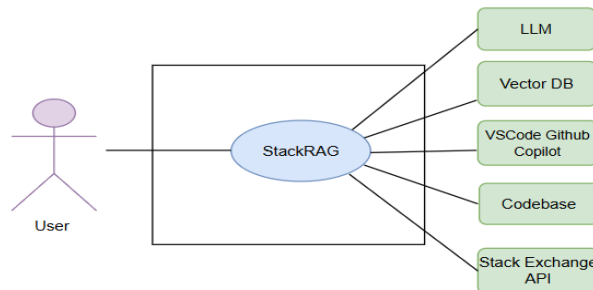


Figure 8: Level 0 (StackRAG)

Use Case Name: StackRAG System

Use Case ID: 0

Primary Actors: User

Secondary Actors: LLM, Vector DB, Codebase, VSCode Github Copilot, Stack Exchange API

Goal in Context: The diagram above illustrates the complete system overview for StackRAG, a tool designed for helping developers for code generation, refactoring and bug fixing.

Description: This is the high level view or level 0 view of the stackRAG system. It has one primary actor called user. The secondary actors are LLM, Vector DB, Codebase, VSCode Github Copilot, Stack Exchange API.

3.1.1 Level 1:

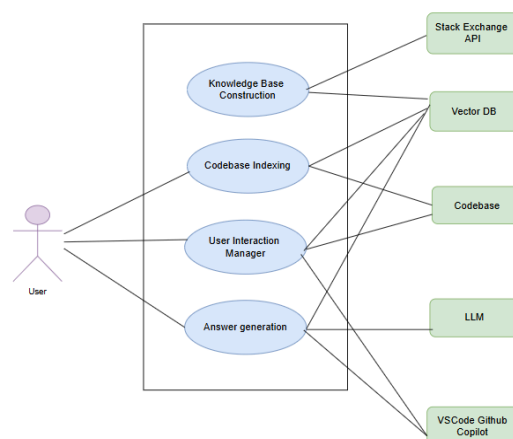


Figure 9: Level 1 (Modules of StackRAG)

Use Case Name: Modules of StackRAG

Use Case ID: 1

Primary Actors: User

Secondary Actors: LLM, Vector DB, Codebase, VSCode Github Copilot, Stack Exchange API

Goal in Context: The diagram above shows the modules of StackRAG. It consists of 5 modules which are further elaborated below.

Description:

1. User Interaction Manager

- **Functionality:** this module manages all sorts of user interaction. Like activating the extension, preparing the codebase indexing, previous interactions.
- **Purpose:** activates the extension, manages user interaction with the extension.
- **Key Role:** Acts as the starting point for all system operations.

2. Knowledge Base Construction

- **Functionality:** this module builds the knowledge base of the system. It consists of three parts: searching from stack overflow, indexing the project and building BM25 reranker.
- **Purpose:** builds the knowledge base of the system with stack overflow posts.
- **Output:** we can retrieve the embedded knowledge of stack overflow.

3. Codebase Indexing

- **Functionality:** Gathers relevant code snippets from the user's project. This is crucial for providing context-aware solutions. This module builds the codebase indexing.
- **Purpose:** By understanding the broader code context, this module enables more accurate solutions.
- **Key Dependencies:** Works closely with the **Code Base** and **Vector DB** to ensure all suggestions are related to the specific code environment.

4. Answer Generation

- **Functionality:** this module is responsible for generating solutions. Solutions include: chat response, bug fixing, code refactoring.
- **Purpose:** generates the solution.
- **Key Dependencies:** depends on knowledge base, indexed codebase and llms.

3.1.1 Level 1.1:

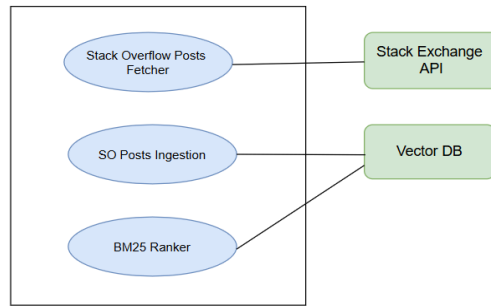


Figure 10: level 1.1 (knowledge base system)

Use Case Name: knowledge base system

Use Case ID: 2

Primary Actors: No primary actor

Secondary Actors: Vector DB, Stack Exchange API

Goal in Context: The goal of this system is to build a knowledge base.

Description:

1. **Stack Overflow Posts Fetcher:** it fetches Stack Overflow Posts by using Stack Exchange API.
2. **SO Posts Ingestion:** It ingests the fetched posts.
3. **BM25 Ranker:** it builds BM25 reranker to optimize the SO posts fetching from vector db later.

3.1.1 Level 1.2:

Use Case Name: codebase indexing system

Use Case ID: 3

Primary Actors: User

Secondary Actors: Vector DB, Codebase

Goal in Context: indexes open codebase on demand.

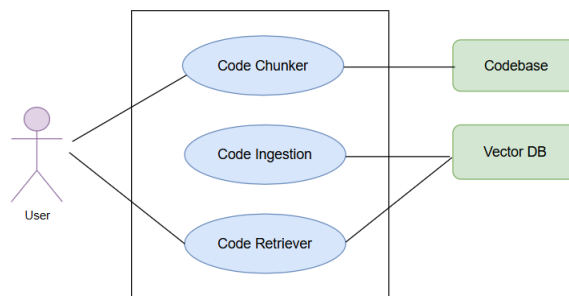


Figure 11: level 1.2 (codebase indexing system)

Description:

1. **Code Chunker:** it chunks the py and js code files based on statement and function types.
2. **Code Ingestion:** it ingests the chunked codes into vector db.
3. **Code Retriever:** the retriever part retrieves the indexed code from the vector db.

3.1.1 Level 1.4:

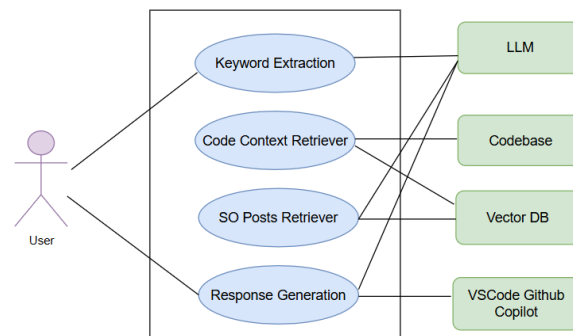


Figure 12: level 1.4 (answer generation system)

Use Case Name: Answer Generation System

Use Case ID: 4

Primary Actors: User

Secondary Actors: LLM, Vector DB, Codebase, VSCode Github Copilot

Goal in Context: it is responsible for generating response.

Description:

1. **Keyword Extraction:** it extracts keywords from the given user query.
2. **Code Context Retriever:** it retrieves the relevant codes from vector db.
3. **SO Posts Retriever:** it retrieves the relevant SO posts from vector db.
4. **Response Generation:** it generates responses using retrieved relevant information with the llm.

3.1.1 Level 1.4.3:

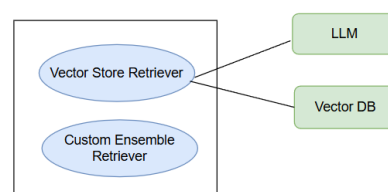


Figure 13: level 1.4.3 (SO posts retriever system)

Use Case Name: SO posts retriever system

Use Case ID: 5

Primary Actors: User

Secondary Actors: LLM, Vector DB

Goal in Context: it is responsible for retrieving SO posts efficiently from vector db.

Description:

1. **Vector Store Retriever:** it retrieves SO posts using vector store retriever. This retriever retrieves SO posts from vector db.
2. **Custom Ensemble Retriever:** it achieves similar performance like langchain's previous vector store retriever.

4. Data Based Modeling

No relational database system is used for this tool. Only VectorDB is used to store the required structured collection. Below is the structure of the documents.

Documents	Attributes	Types
Question	page_content	TEXT
	Type (metadata)	TEXT
	Link (metadata)	TEXT
	Score (metadata)	INTEGER
	Tags (metadata)	ARRAY[VARCHAR(50)]
	Question_id (metadata)	INTEGER
	Answer_count (metadata)	INTEGER
	View-count (metadata)	INTEGER
Answer	page_content	TEXT
	Type (metadata)	TEXT
	Question_id (metadata)	INTEGER
	Score (metadata)	INTEGER
Code	page_content	TEXT
	filePath (metadata)	VARCHAR(255)
	Type (metadata)	VARCHAR(30)
	Extension (metadata)	VARCHAR(10)

Table-1: Schema table

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

a) Models b) keywordExtractor c) Retriever d) CustomEnsembleRetriever e) Chat
f) AnswerGeneration g) Ingestion h) BuildAndSaveBM25 i) JSChunker j) PYChunker
k) CodeIngestion l) CodebaseIndexManager m) VSCodeExtensionManager

5.2 Class Cards

Models	
Attributes	Methods
- ollama_embeddings - ollama_chat - mistral_embeddings - mistral_chat	+ __init__()
Responsibilities	Collaborators
Define the chat and embedding models	• AnswerGeneration • Retriever

Table-2: CRC card (**Models**)

KeywordExtractor	
Attributes	Methods
- keyword_extractor_prompt - extracted_keywords - keyword_chain	+ extract_keywords()
Responsibilities	Collaborators
Extract the keywords using the keyword extracting prompt.	• AnswerGeneration

Table-3: CRC card (**KeywordExtractor**)

Retriever	
Attributes	Methods
- bm25_retriever - vector_retriever - ensemble_retriever - questions - so_discussions	+ retrieve()
Responsibilities	Collaborators
Retrieves related SO posts using both bm25 and vector retriever.	• CustomEnsembleRetriever • AnswerGeneration • BM25Builder • Models

Table-4: CRC card (**Retriever**)

CustomEnsembleRetriever	
Attributes	Methods
- retrievers - weights	+ __init__() + get_relevant_documents()
Responsibilities	Collaborators
Define a custom ensemble retriever	• Retriever

Table-5: CRC card (**CustomEnsembleRetriever**)

AnswerGeneration	
Attributes	Methods
- answer_prompt - answer_chain - response	+ generate_chat_response() + fix_bug() + refactor_code()
Responsibilities	Collaborators
Generates responses to users' requests.	• Models • Retriever • Keyword Extractor • VSCodeExtensionManager

Table-6: CRC card(**AnswerGeneration**)

SOPostsFetcher	
Attributes	Methods
- state - page - accepted_qs - accepted-answer_ids - answers - qna_pairs	+ fetch_all_questions_with_accepted_answers() + fetch_all_questions_with_no_accepted_answers() + clean_html_content() + load_state() + save_state() + save_questions() + chunk_list()
Responsibilities	Collaborators
Fetches posts from Stack Overflow	• Ingestion

Table-7: CRC card(**SOPostsFetcher**)

Ingestion	
Attributes	Methods
- text_splitter - embeddings - qa_collection - qa_threads - SO_collections - documents	+ main() + thread_to_document() + process_documents() + store_in_vectordb()

Responsibilities	Collaborators
Ingest the fetched SO posts into the vector database.	• SOPostsFetcher • BM25Builder

Table-8: CRC card(**Ingestion**)

BM25Builder	
Attributes	Methods
- raw_collection - all_docs - question_docs	+build_and_save_bm25()
Responsibilities	Collaborators
Build and save the bm25 reranker for future use.	• Retriever • Ingestion

Table-9: CRC card(**BuildAndSaveBM25**)

JSChunker	
Attributes	Methods
- workspace_path - jsFiles - chunks	+ findJsFiles() + splitJsCodes() + readJsFilesConcurrently() + processJsFiles()
Responsibilities	Collaborators
Chunks the javascript codes into function, statement level.	• CodebaseIndexManager • CodeIngestion

Table-10: CRC card(**JSChunker**)

PYChunker	
Attributes	Methods
- workspace_path - pythonFiles - chunks	+ findPythonFiles() + splitPythonCodes() + readPythonFilesConcurrently() + processPythonFiles()

Responsibilities	Collaborators
Chunks the python codes into function, statement level.	• CodebaseIndexManager • CodeIngestion

Table-11: CRC card(**PYChunker**)

CodeIngestion	
Attributes	Methods
- text_splitter - embeddings - collection - threads - code_collections - documents	+ main() + thread_to_document() + process_documents() + store_in_vectordb()
Responsibilities	Collaborators
Ingest chunked codes into a vector database.	• CodebaseIndexManager • JSChunker • PYChunker

Table-12: CRC card(**CodeIngestion**)

CodebaseIndexManager	
Attributes	Methods
- embeddingModel - collectionName	+ readIndex() + createIndex() + updateIndex() + deleteIndex()
Responsibilities	Collaborators
Manages the crud operations related to codebase index.	• CodebaseIndexManager • JSChunker • PYChunker • VSCodeExtensionManager

Table-13: CRC card(**CodebaseIndexManager**)

VSCodeExtensionManager	
Attributes	Methods
- webViewView	+ activate_extension() + showInwebview() + getWebviewController() + callCopilot()
Responsibilities	Collaborators
Manage the user interface within VSCode, including displaying chatbox, options for bug fixing, code refactoring, history, codebase indexing.	• CodebaseIndexManager • AnswerGeneration

Table-14: CRC card(VSCodeExtensionManager)

5.3 CRC Diagram

The CRC diagram of the classes is given below:

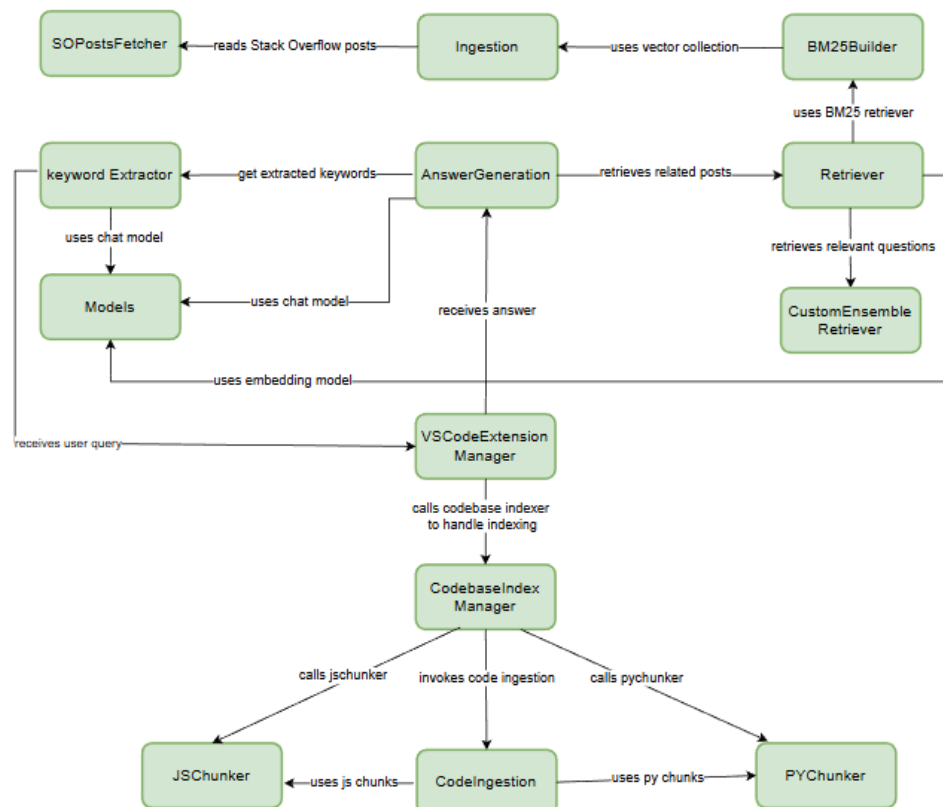


Figure 14: [CRC Diagram of StackRAG](#)

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

At the architectural design level, a software architect uses an architectural context diagram (ACD) to model how software interacts with entities external to its boundaries. Systems that interoperate with the target system are represented as -

- **Superordinate systems**— Those systems that use the target system as part of some higher-level processing scheme.
- **Subordinate systems**— Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.
- **Peer-level systems**— Those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system)
- **Actors**— Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.

Each of these external entities communicates with the target system through an Interface. Representation of StackRAGin Architectural Context Diagram is delineated as follows:

- **Superordinate systems** - Our system does not have any superordinate system.
- **Subordinate systems** - Our system uses the **LLM** to generate answers, **Stack Overflow Posts** as an external knowledge source, **Codebase** for context and **Vector DB** for storing data.
- **Peer-level systems** - Our system has one peer-level system that is **VSCode Github Copilot**.
- **Actors** - **Developers** are the core users of this system.

The architectural context diagram for the tool is given below.

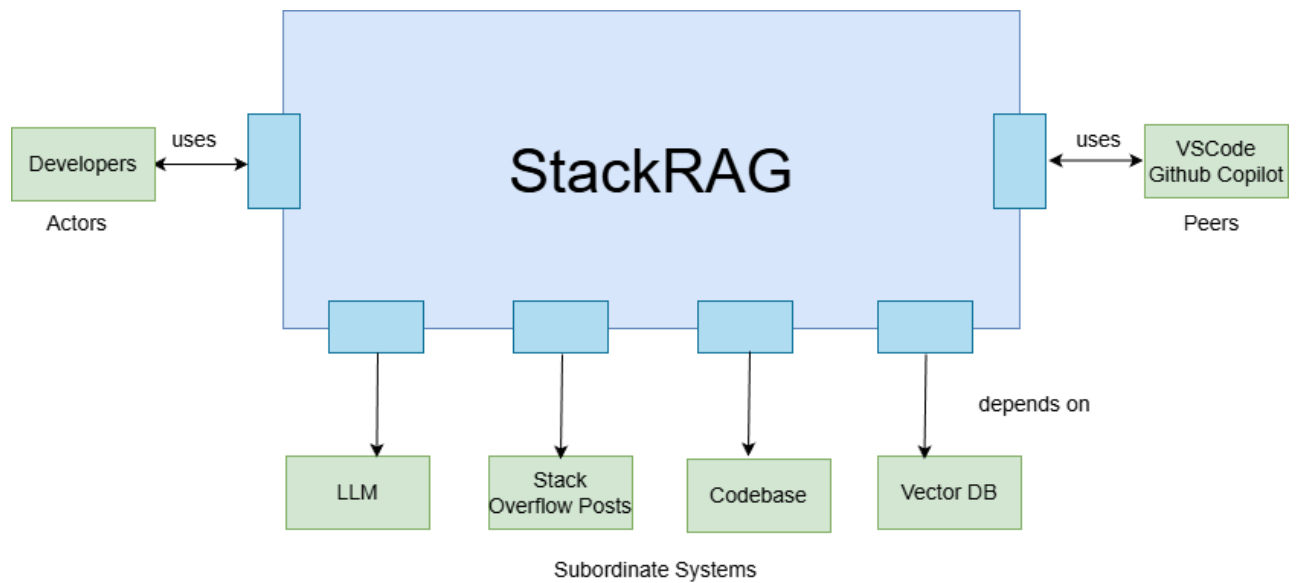


Figure 15: Architectural Context Diagram

6.2 Archetypes

The layered architecture:

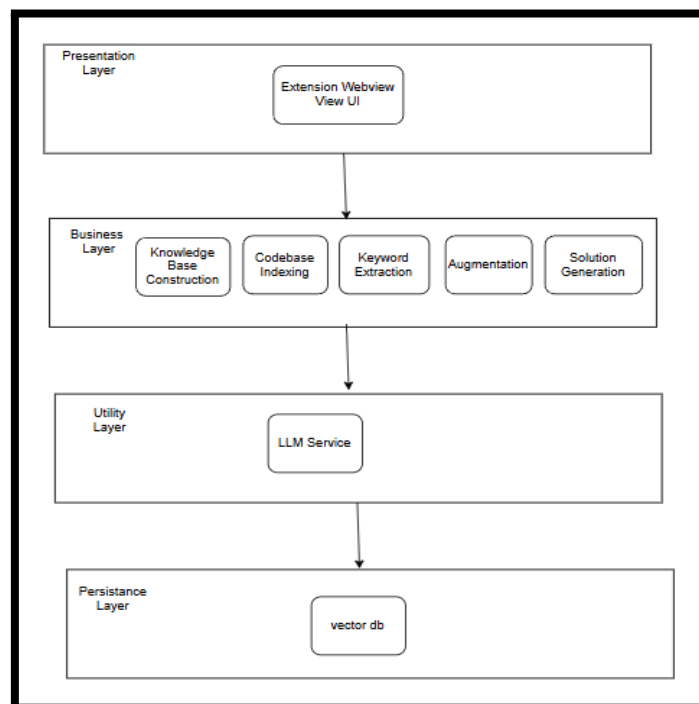


Figure 16: Layered Architecture

Description:

1. Presentation Layer
 - Extension Webview View UI: it handles the GUI part of the tool.
2. Business layer
 - Knowledge base construction: it builds the knowledge base for the system.

- Codebase indexing: it indexes the codebase on demand.
 - Keyword extraction: it extracts the keyword from the user query.
 - Augmentation: it augments the retrieved relevant information with the user query.
 - Solution generation: it is responsible for generating the solution using llm.
3. Utility layer
 - Llm service: the tool uses llm service as a utility to generate the final solution.
 4. Persistence layer
 - Vector db: it stores our knowledge base and codebase indexing.

6.3 Top Level Components

The overall architectural structure with top-level components is illustrated below

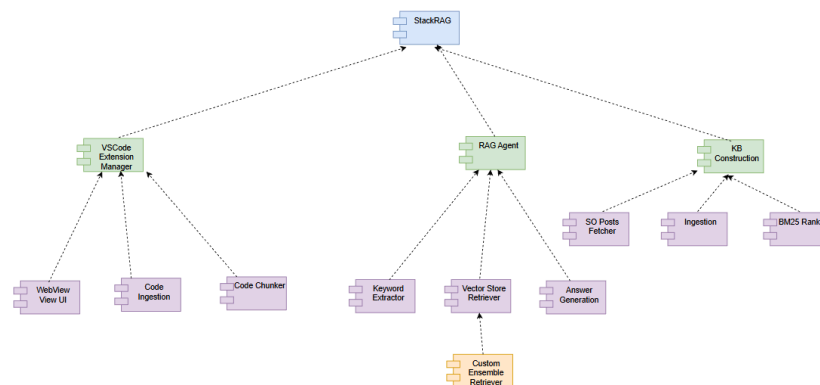


Figure 18: Top Level Components

6.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown below:

Requirements	Components
Integration into VSCode	Webview View UI
Interactive chat	BM25 Ranker, Keyword Extractor, Vector Store Retriever, Custom Ensemble Retriever, Answer Generation
Bug fixing	BM25 Ranker, Keyword Extractor, Vector Store Retriever, Custom Ensemble Retriever, Answer Generation
Code refactoring	BM25 Ranker, Keyword Extractor, Vector Store Retriever, Custom Ensemble Retriever, Answer Generation

Codebase indexing	Code Ingestion, Code Chunker, Codebase Indexer
Codebase Summary	Code Ingestion, Code Chunker, Codebase Indexer
Appending links of related Stack Overflow Posts	Vector Store Retriever, Custom Ensemble Retriever
Side by side view of StackRAG and VSCode github copilot response	Webview View UI

Table-15: Requirements Mapping with components

6.5 Deployment Diagram

The deployment diagram of StackRAG tool is given below:

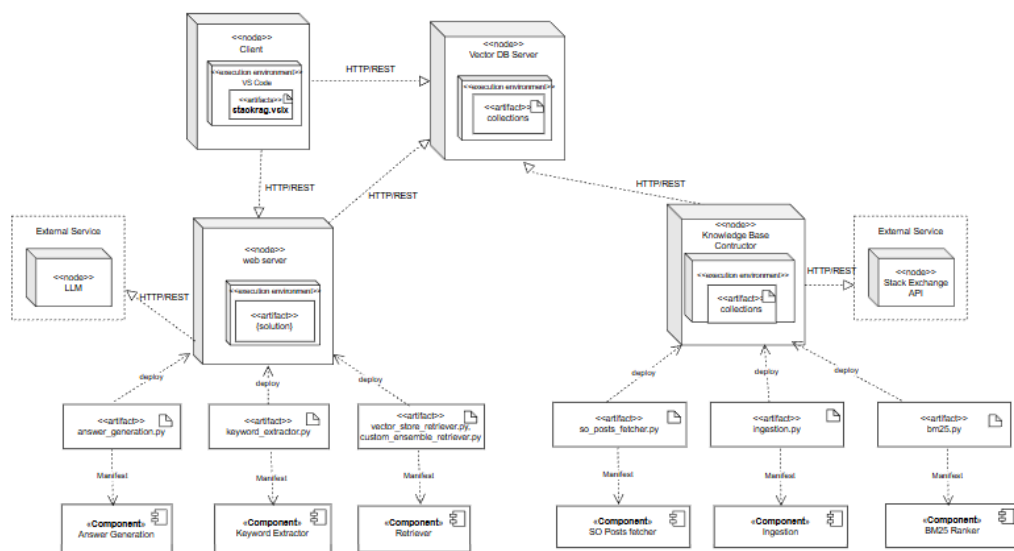


Figure 21: [Deployment diagram](#)

7. Methodology

This chapter explains the workflow, component level design and their detailed implementation, elaboration of the deployment diagram of the STackRAG tool.

7.1 Workflow diagram

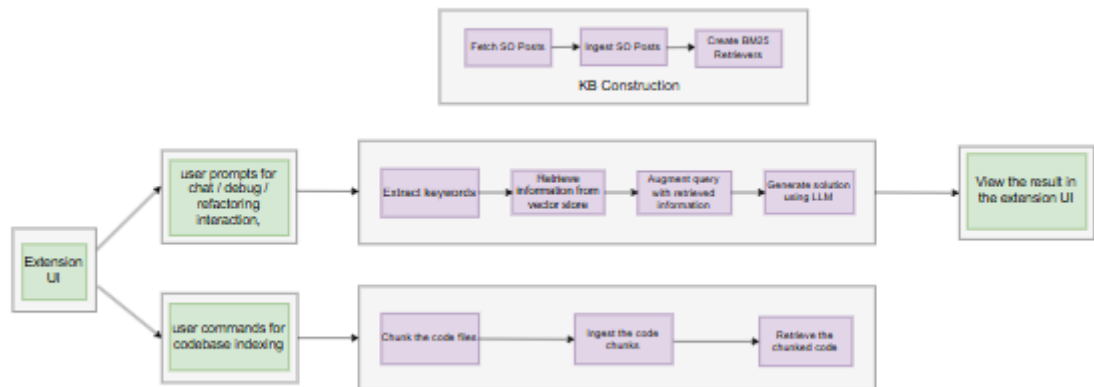


Figure 20: workflow diagram of stackrag

7.2 knowledge Construction

7.2.1 Fetch SO posts

This is the very first step of the stackrag tool. In this part, we fetch the posts from stack overflow. According to the requirements, posts for the python and javascript languages are fetched from the site. Total **1,80,000+ python** related posts and **1,25,000+ javascript** related posts are fetched. By utilizing the stack exchange api key, all of these posts are fetched, cleaned and stored in separate json files locally.

7.2.2 Ingest SO posts

This is responsible for ingesting the fetched SO posts in a chroma db collection. At first, it reads the python related posts from the respective json file and then it chunks the posts using the **recursive splitter library** offered by langchain framework. After chunking it embedded those chunks and stored them in a collection in chroma db. Then the same process continues for fetched javascript posts and stored in the collection dedicated for javascript posts.

7.2.3 Create BM25 retriever

It helps us achieve the optimization during the retrieval time. It creates a bm25 retriever prior to any operation. Even when the backend spins up, it is the first thing the backend loads. It creates separate **pickle files** for python and javascript languages for bm25 retriever. The pickle files are stored locally.

7.3 Output Generation

7.3.1 Extract keywords

It is the first step of our RAG architecture. It is responsible for generating keywords from the user prompt. It utilizes the **prompt engineering** to generate the keywords by using the power of LLM. it generates the list of extracted keywords from the prompt.

7.3.2 Retrieve relevant information from kb

After generating the keywords, this step is executed to retrieve the relevant posts with respect to the extracted keywords from the knowledge base. If the query is related to python, then information retrieved from the python collection, otherwise from the javascript collection. For retrieving the information from the collection, it uses **ensemble retrievers** of two retrievers. One is a **bm25 retriever** and another is a **vector store retriever**. BM25 retriever works best for questions or keywords whereas vector store retriever works best for code. As we need to retrieve the posts mostly based on the keyword not the code, that's why the ensemble retriever gives more score to bm25 retriever and less to vector store retriever.

7.3.3 Augmentation

This is responsible for augmenting the **LLM query** with the **retrieved posts** and user passed **contexts**. After augmenting, we get the ready prompt which will be passed to the LLM.

7.3.4 Generate solution

This is the last step of the RAG architecture of the stackrag tool. In this step, we pass the prompt to the **LLM**. LLM processes the prompts and generates solutions by using the retrieved information and its own knowledge. After generating the solution, the user can view the generated solution in the extension UI.

7.4 Codebase Indexing

7.4.1 Chunks the code files

It chunks the codefiles in a codebase or project in two categories. The chunks can be of statements or functions. It processes the javascript and python files. Then it chunks the Code file in two types. The method (**def or function**) block is chunked as functions. The other part of the code is chunked as a statement. Like the variable declaration or statements beyond the function.

7.4.2 Ingest the code chunks

It ingests the code chunks and stores them into **qdrant db** remotely.

7.4.3 Retrieve the related code chunks

It is responsible for retrieving the related code chunks based on the **user query** from the qdrant db.

8. Component Level Design

This chapter describes the component level design for the tool. Component level design occurs after the first iteration of architectural design has been completed.

8.1 Identify design classes

When an object-oriented software engineering approach is chosen, the component level design focuses on the elaboration of problem domain-specific classes and the definition and refinement of infrastructure classes contained in the requirements model. The Identified Design Classes Are –

a) SOPostsFetcher b) Ingestion c) BM25Retriever d) keywordExtractor e) Retriever
f) CustomEnsembleRetriever g) AnswerGeneration h) JSChunker i) PYChunker
j) CodeIngestion k) IndexService

8.2 Persistent data sources

8.2.1 Databases and files

In this tool, only vector databases are used. Chroma db and quadrant db are used as vector databases.

At the time of building knowledge, after collection of the posts, it primarily stores them in json files.

8.2.2 Identify design classes of persistent data sources

There is no specific class that dedicatedly handles the databases. But to store and interact with the databases, the Ingestion, CodeIngestion, BM25Retriever and Retriever classes are used.

8.3 Elaboration of behavioral representations of design classes

File: [indexService.ts](#)

It initializes all the required environment variables. Then it defines the text splitter. It can perform the deletion operation of a collection. Also it can index a codebase and retrieve relevant code chunks.

File: jsChunker.ts

1. findJsFiles(rootDir)

It searches for js files in the codebase by recursively traversing. Then It splits the code files into chunks of two types. Statement and function. It processes files in batches. Thus it ensures time optimization during indexing.

File: [pyChunker.ts](#)

It searches for py files in the codebase by recursively traversing. Then It splits the code files into chunks of two types. Statement and function. It processes files in batches. Thus it ensures time optimization during indexing.

File: codeIngestion.ts

It loads environment configuration. Then reads the json file. After that it converts the posts into chunks. The chunks are embedded and then stored in a vector db.

File: custom_ensemble_retriever.py

It reuses the langchain behavior for retrieving.

File: vector_store_retriever.py

It takes the collection db name, the query string and the language. Then it uses the custom ensemble class that is defined and retrieves the relevant document.

File: [bm25.py](#)

It creates a bm25 retriever and stores it into a pickle file.

File: ingestion.js

It loads environment configuration. Then reads the json file. After that it converts the posts into chunks. The chunks are embedded and then stored in a vector db.

File: so_posts_fetcher.py

It loads the necessary variable from the .env file. It checks for previously saved fetch state (last page number). Then it continue sto fetch posts. It parse HTML, extracting text while preserving <code> content. Whitespace is normalized and the function returns clean text strings suitable for storage and indexing.

File: keyword_extractor.py

It accepts a user query string as the question. Then It configures the prompt to extract the keywords. It invokes the llm to extract keywords. It receives the LLM's extracted keywords as a string.

File: chat.py

It takes a user query (question) and the programming language (python or javascript). It calls the extracting keyword step. It retrieves the relevant pattern. Similar to chat , it utilizes the chat prompt to fix the bug with the user prompt and retrieved documents.

File: bug_fixing.py

It takes a user query (question) and the programming language (python or javascript). It calls the extracting keyword step. It retrieves the relevant pattern. Similar to chat , it utilizes the bug fix prompt to fix the bug with the user prompt and retrieved documents.

File: code_refactoring.py

It takes a user query (question) and the programming language (python or javascript). It calls the extracting keyword step. It retrieves the relevant pattern. Similar to chat , it utilizes the code refactor prompt to fix the bug with the user prompt and retrieved documents.

8.4 Elaboration of deployment diagram

During component-level design, deployment diagrams can be elaborated to represent the location of key packages of components. The specific hardware and operating system environment(s) that will be used is (are) specified and the location of component packages within this environment is indicated.

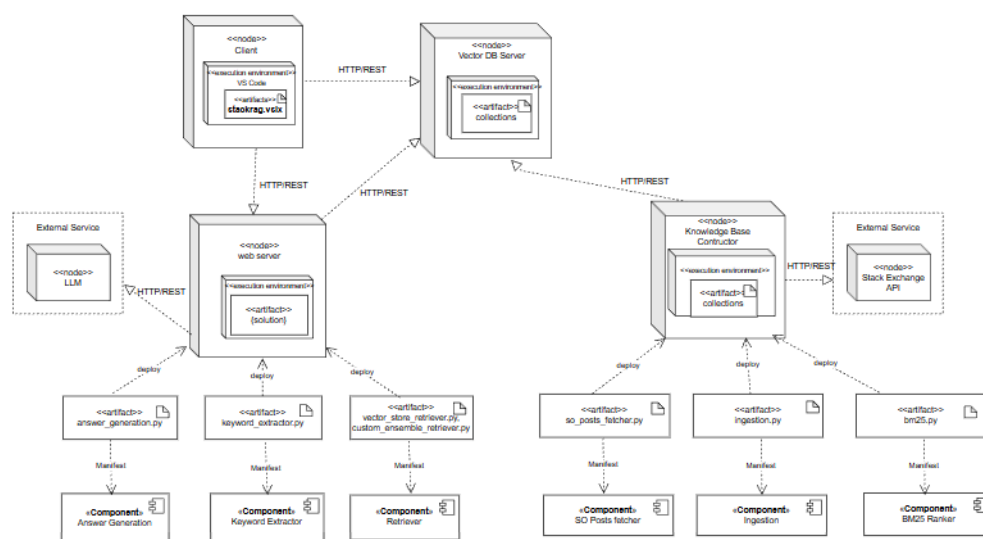


Figure 21: [Deployment diagram](#)

The StackRAG tool has mainly **three** components.

1. **Extension UI:** StackRAG VSCode extension is hosted on the vscode marketplace. It is the client part. It contains all of the artifacts related to UI. For example: [chatWebView.ts](#), [chatSidebarProvider.ts](#), [extension.ts](#), [pyChunker.ts](#), [jschunker.ts](#), [codeIngestion.ts](#), [indexService.ts](#).
2. **Extension backend:** it is the backend web server of the tool. This is a python server implemented using **fast api**. It contains all of the artifacts related to business logic. For example: [keywordExtractor.py](#), [retriever.py](#),

[customEnsembleRetriever.py](#), [bug_fixing.py](#), [chat.py](#), [code_refactoring.py](#), [codebase_summary.py](#).

3. **KB:** It is the heart of the tool. It generates the knowledge base of the tool and hosts it in the **hugging face space**. The artifacts are: [so_posts_fetcher.py](#), [ingestion.js](#), [bm25.py](#).

8.5 Code implementation

Section 1: STACKRAG/ (VS Code Extension)

File: `extension.ts`

Details:

It at first registers commands and event listeners when extension loads. Then it initializes the chat webview and indexing services. After that it sets up communication between extension and webview.

File: `indexing/codeIngestion.ts`

Details:

It ingests the code files chunks in the quadrant vector db.

File: `indexing/indexService.ts`

Details:

It performs three operations. It runs the function that is responsible for storing the index in vector db. It deletes any collection for the vector db. It reindexes any codebase.

File: `indexing/jsChunker.ts`

Details:

It at first parses js or ts code using AST. then it retrieves the file contents. After that it chunks them in two categories. Statement and functions.

File: `indexing/pyChunker.ts`

Details:

It at first parses py code using AST. then it retrieves the file contents. After that it chunks them in two categories. Statement and functions.

File: `view/chatWebView.ts`

Details:

It creates an HTML webview view in VS Code. It handles all of the interaction between the html file and the backend. It is the sidebar provider.

File: view/webView.html

Details:

The html file handles user input from the chat interface. It communicates with the extension backend. It also displays AI responses and code suggestions.

Section 2: BACKEND

File: app.py

Details:

It initializes the fast api application. It sets up the endpoints.

File: bug_fixing_prompt.py, chat_prompt.py, code_refactoring_prompt.py, codebase_summary_prompt.py, keyword_extractor_prompt.py

All of these files contain prompts for specific tasks.

File: bug_fixing.py

Details:

It at first extracts the keywords. Then it retrieves relevant code from vector store. Then it augments the prompt and passes the prompt to the llm. It returns the llm response.

File: chat.py

Details:

It at first extracts the keywords. Then it retrieves relevant code from vector store. Then it augments the prompt and passes the prompt to the llm. It returns the llm response.

File: code_refactoring.py

Details:

It at first extracts the keywords. Then it retrieves relevant code from vector store. Then it augments the prompt and passes the prompt to the llm. It returns the llm response.

File: codebase_summary.py

Details:

It receives the input from the user. Then it constructs the prompt to generate codebase summary with the relevant information. Then it passes the prompt to the llm. It returns the response generated by llm.

File: custom_ensemble_retriever.py

Details:

It takes a query. Then it performs semantic search on vectors and BM25 keyword search. Then it merges results with weighted scoring. After that it returns top-k combined results.

File: keyword_extractor.py

Details:

It receives user query. Then it calls LLM with keyword extraction prompt. After receiving the results, it parses and cleans extracted keywords. Then it returns the extracted keywords .

File: vector_store_retriever.py**Details:**

It takes query string. Then it embeds query using embedding model. It can perform cosine similarity search. After that returns results.

File: pickle_loader.py

Purpose: Loads pickled objects from disk storage.

Section 3: KB (Knowledge Base Ingestion)

File: bm25.py

Purpose: BM25 search implementation for keyword-based retrieval.

File: ingestion.js**Details:**

It ingests the chunked SO posts in a chroma db vector database in a batch of size 200.

File: so_posts_fetcher.py**Details:**

It is responsible for fetching the stack overflow posts using the stack exchange api. It at first fetches the posts. Then it cleans them. Then it stores them in a json file.

Folder: constants**Key files:**

[collections.py](#): Collection definitions, [file_paths.py](#): File path configurations, [urls.py](#) API endpoints and URLs, [sites.py](#) : Supported websites, [tags.py](#) : Tag configurations, [vector_store.js](#) : Vector store configuration

9. User interface design

User interface design creates an effective communication medium between a human and a computer. Following a set of interface design principles, the design identifies interface objects and actions and then creates a screen layout that forms the basis for a user interface prototype.

Interface Analysis

In the case of user interface design, understanding the problem means understanding:

- The people (end users) who will interact with the system through the interface.
- The tasks that end users must perform to do their work.
- The content that is presented as part of the interface and
- The environment in which these tasks will be conducted.

In the sections that follow, we examine each of these elements of interface analysis with the intent of establishing a solid foundation for the design tasks that follow.

9.1 User Analysis

Any developer who uses vscode, can use this tool for their python and javascript language related tasks.

9.2 Task Analysis

9.2.1 User Task: Open VSCode Extension

- Launch **VSCode** and *open* the **Extensions tab**

9.2.2 User Task: Search StackRAG & Install

- *Search* for **StackRAG**
- *Click* install in the **StackRAGExtension Page**.

9.2.3 User Task: Open Any Python or Javascript Project

- *Opens* a **Python or javascript project** in VSCode

9.2.4 User Task: Open StackRAG Sidebar

- *Click* on the **StackRAG icon** to open the sidebar

9.2.5 User Task: select mode to interact

- *Click* on **mode option** to open the interaction mode.

9.2.6 User Task: enable the comparison

- *Click* on **enable comparison checkbox** to receive response from both of the stackrag tool and vscode github copilot.

9.2.7 User Task: choose to index the codebase

- *Click* on the **index option** to index the codebase to have a better response.

9.2.8 User Task: type the prompt in the chat bar

- *type* the prompt in the **chat bar**.
- Or *select* one from the **suggested questions**.

9.2.9 User Task: add explicitly file context

- *Click* on **add file context icon** to select the files to use as context.
- *Click* on the **add or cancel button** to add or cancel the file context.

9.2.10 User Task: send the query to the tool

- *Click* on the send **button**.

9.2.11 User Task: select one response to continue

- Click on the **select option** of the response the user prefers more.

9.2.12 User Task: copy the code attached with the response

- Click on the **copy option** to copy the code.

9.2.13 User Task: click on the see previous interaction option at the left top

- Click on **see previous interaction** to see the previous interaction.

9.2.14 User Task: click on any date to see previous interaction

- Click on any **date**

9.2.15 User Task: click on any mode to see previous interaction

- Click on any **mode**
- View the **previous interaction**

9.2.16 User Task: click on the codebase summary button

- Click on the **codebase summary button**
- View the **codebase summary**

Objects (boldface) and actions (italics) are extracted from this list of tasks. This representation of objects (boldface) and actions (italics) are also maintained in the next section.

9.3 User Manual

User interface layouts with corresponding tasks, objects and actions and how to interact with them are illustrated below:

9.3.1 VSCode Extension Page

After launching the VSCode extension, the user will click on the Extension icon on the left panel. By this, user performs **task 1: Open VSCode Extension** - Launch **VSCode** and open the **Extensions tab**.

Then user performs the **Task-2: Search StackRAG & Install**. They search by typing stackrag and found the published extension. Then click on the install option to install it.

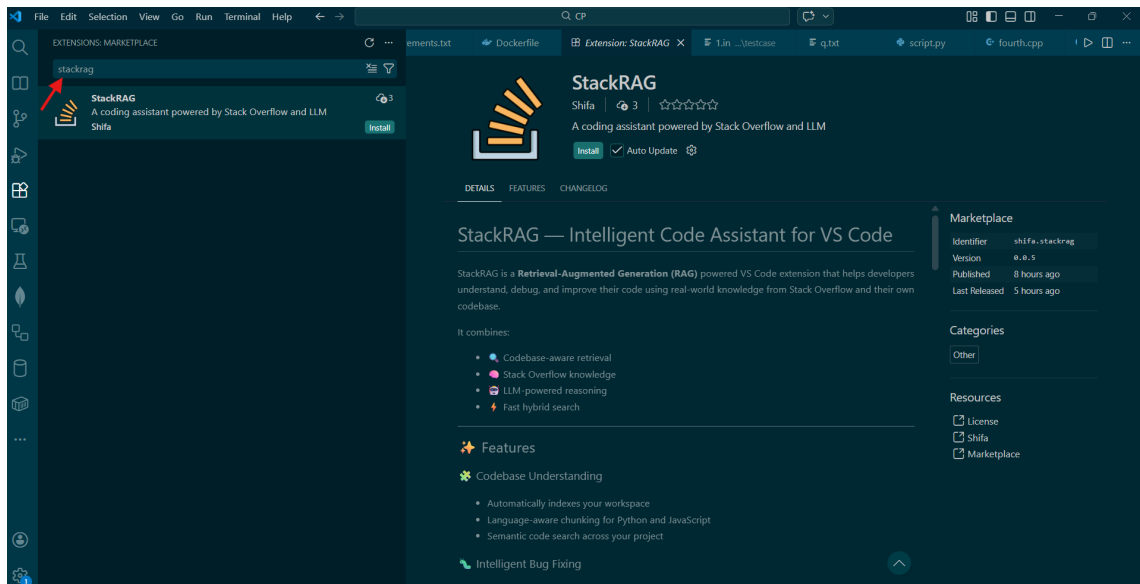


Figure: 22 (extension market place)

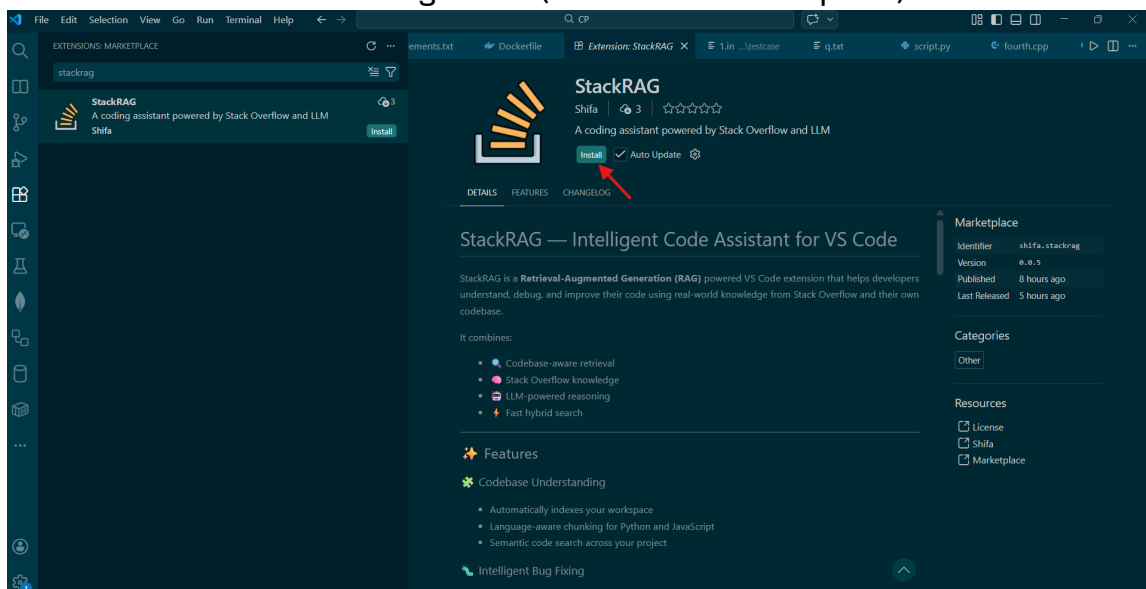


Figure: 23(extension installation)

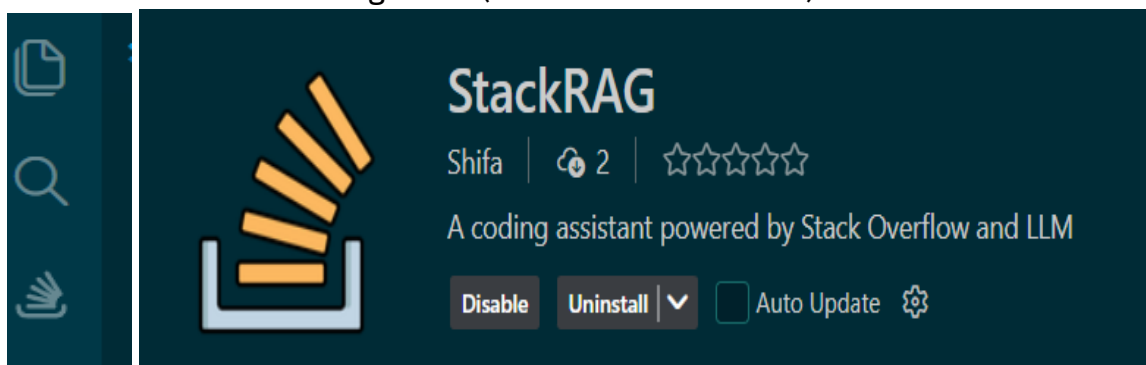


Figure: 24 (installed extension in the activity bar)

9.3.2 Landing page of StackRAG

After installing the extension, if the user opens it, they can see the landing page of the extension. The landing page provides some suggestions and all of the achievable functionalities of the tool. Thus the user performs the **Task-3: Open StackRAG Sidebar**.

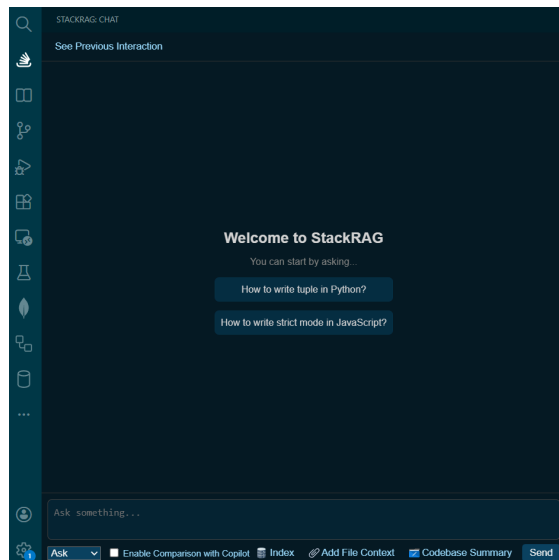


Figure: 25 (the landing page)

9.3.3 Generate solution

To interact with the tool either in chat or debug or refactor mode, the generation solution manual is the same except for changing the mode explicitly. At first, user performs the **Task-4: Open Any Python or Javascript Project** in vscode.

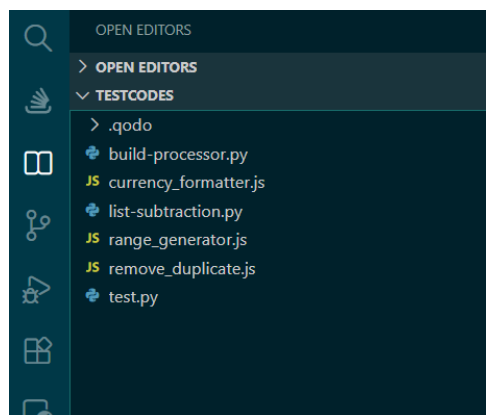


Figure: 26 (a project of python and js)

Then user can perform the **Task-5: select mode to interact** by selecting the interaction mode.

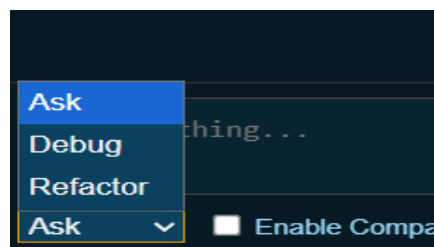


Figure: 27 (mode selection)

Then if user wishes to compare the tool's output with copilot, then they can perform the **Task-6: enable the comparison** by **enabling comparison checkbox** to receive response from both of the stackrag tool and vscode github copilot.

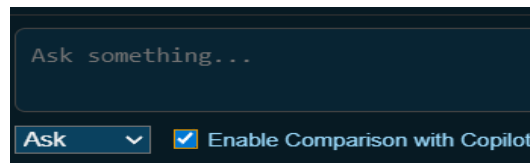


Figure: 28 (enable comparison mode)

Then if the user wants to receive more codebase grounded answers, then they can perform **Task-7: choose to index the codebase** by clicking on the **index option** to index the codebase to have a better response.

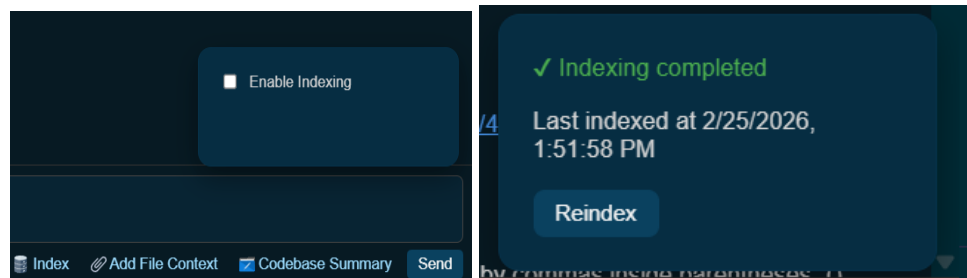


Figure: 29 (enable indexing option and indexing is completed)

Now the user can perform **Task-8: type the prompt in the chat bar**. They can either *type* the prompt in the **chat bar** or *select* one from the **suggested questions**.

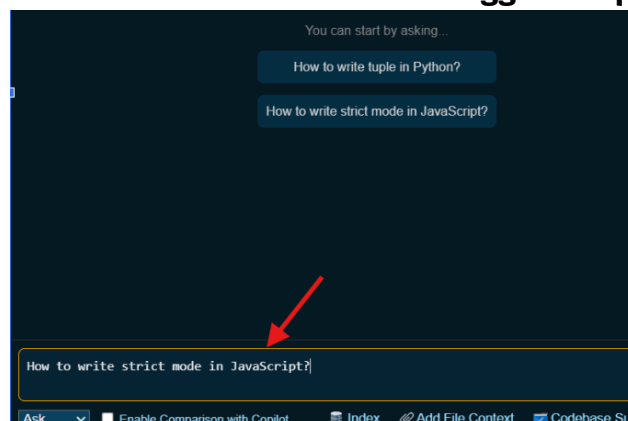


Figure: 30 (typing the prompt)

Also if the user wants to add explicitly the functionality **Task-9: add explicitly file context**, they can perform it. By *Clicking* on **add file context icon** to select the files to use as context. They have two options. They may *Click* on the **add or cancel button** to add or cancel the file context.

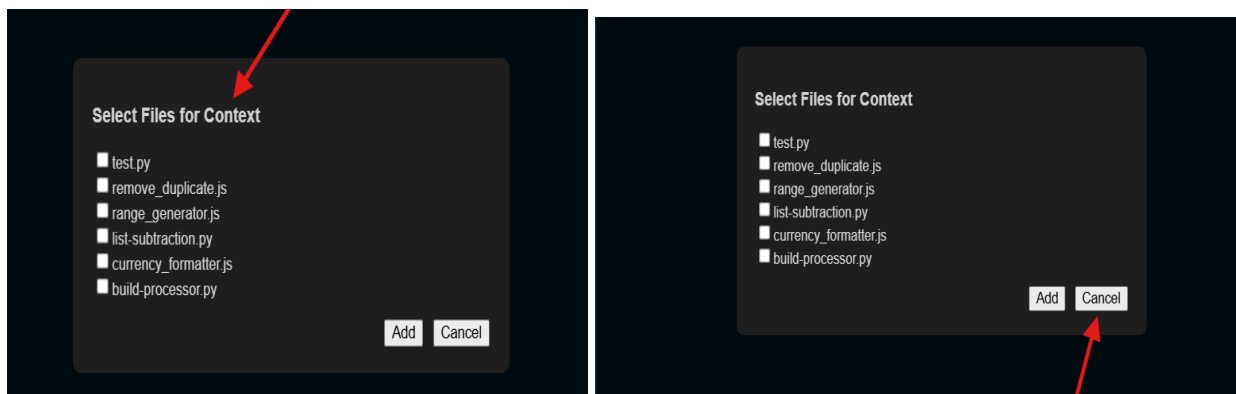


Figure: 31 (selection of file context along with add and cancel button)

After having the configurations prior prompt sending, users now can achieve the **Task-10: send the query to the tool**. Users can click on the send button to send the query to the tool. After sending the query, user will see this type of view.



Figure: 32 (query is sent)

9.3.4 View the response

After receiving the response from the backend, the response will be displayed in the UI. If the user enables the comparison mode, they will see the output in the side by side view (generated by both stackrag tool and vscode copilot). Otherwise, they will see one output generated by the tool. In that case, users will have nothing to select from.

So, if the comparison mode is enabled, users can perform the **Task-11: select one response to continue** by *Clicking* on the **select option** of the response the user prefers more.

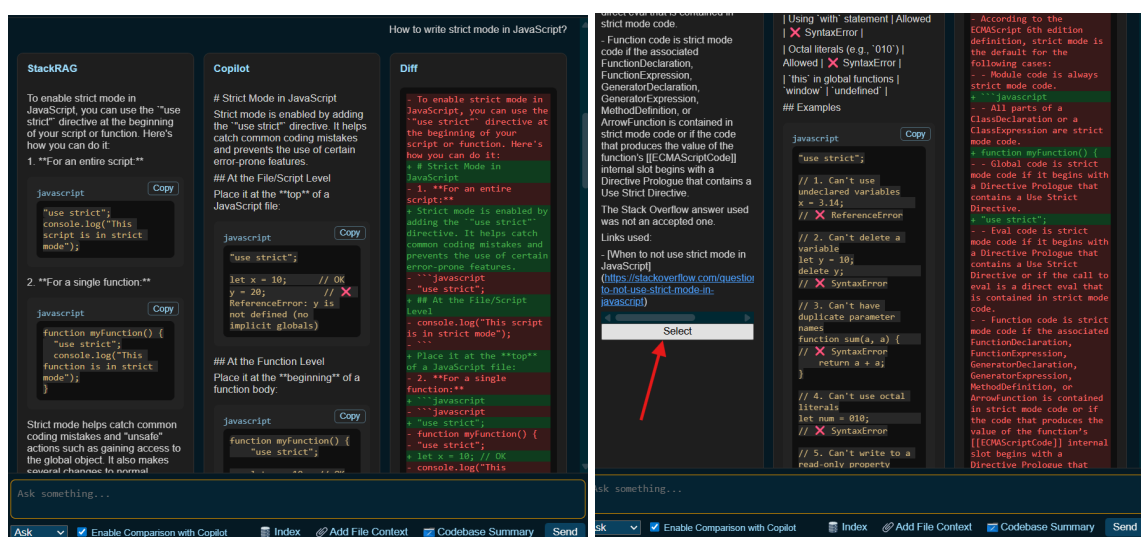


Figure: 33 (side by side output view and selection option)

After selecting one response, the users can see it in the extension UI.

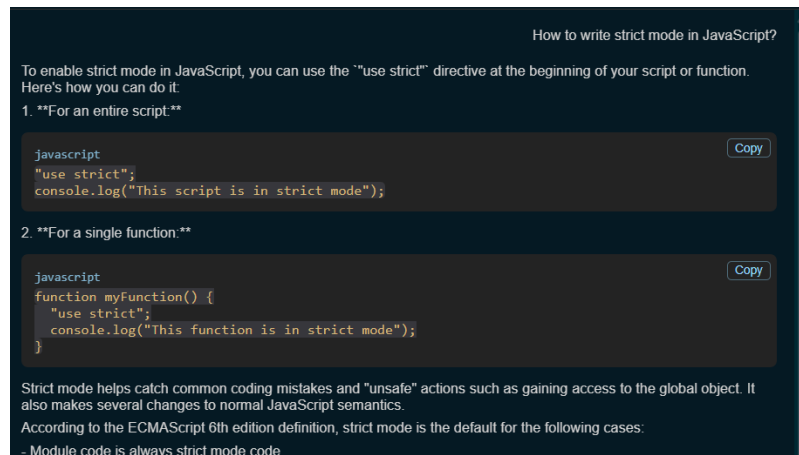


Figure: 34 (selected response)

Now, to use the response, the users can perform the **Task-12: copy the code attached with the response** by *Clicking* on the **copy option** to copy the code.

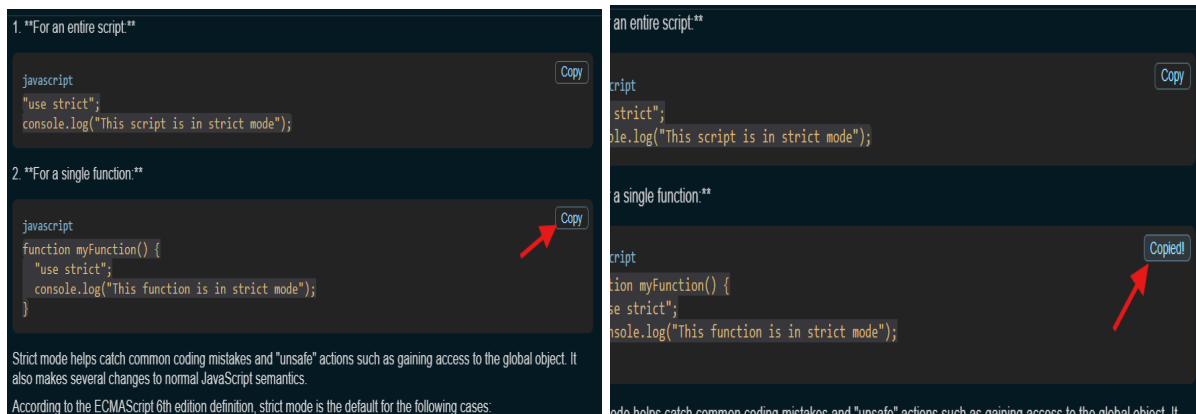


Figure: 35 (copy button and copied the code)

9.3.5 See the previous interactions

If users want, they can view the previous interaction. Thus they perform the **Task-13: click on the see previous interaction option at the left top.**

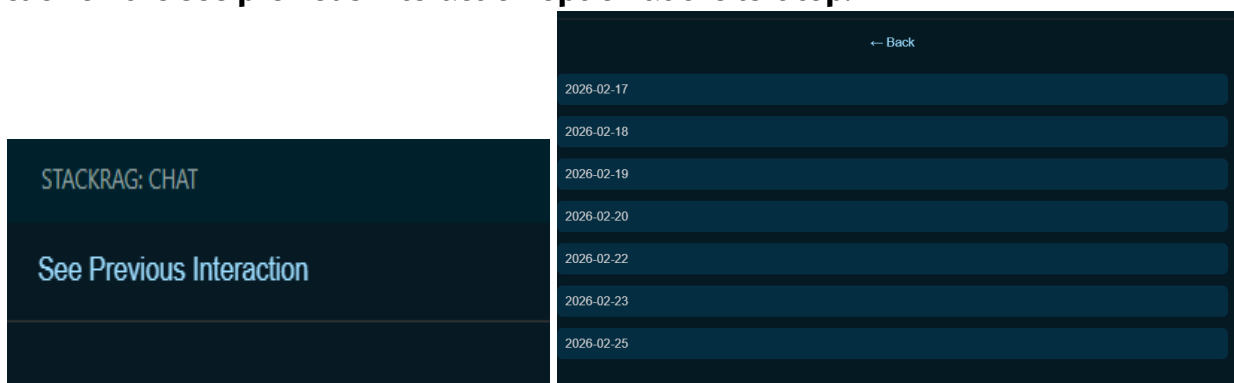


Figure: 36 (see previous interaction option and the datewise interactions)

Now users can perform the **Task-14: click on any date to see previous interaction** by *Clicking* on any **date**.

After the users can perform the **Task-15: click on any mode to see previous interaction** by *Clicking* on any **mode**. After that, they will view the **previous interaction**.



Figure: 37 (mode selection and view the previous interaction)

9.3.6 Generate codebase summary

Users can have the summary of the open codebase and perform the **Task-16: click on the codebase summary button** by clicking on the **codebase summary button**. Then they will see that the prompt for codebase indexing is auto inserted in the input bar. Then by clicking on the **send button**, users can view the **codebase summary**.



Figure: 38 (codebase summary option and input auto filled with query)

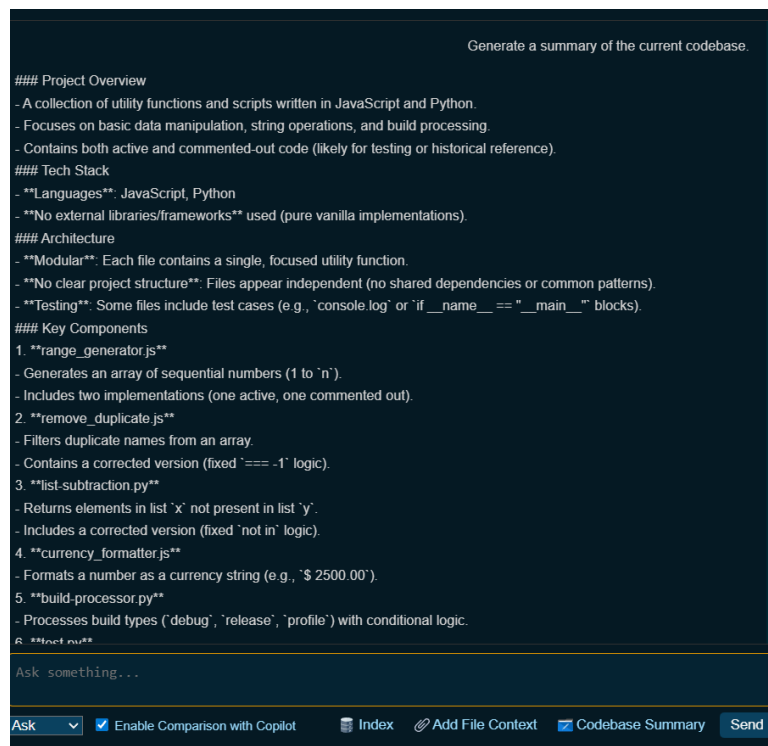


Figure: 39 (codebase summary)

10. Preliminary Test Plan

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

10.1 High-Level Description of Testing Goals

Testing goals for this tool are represented below from a high level-

- To verify that StackRAG meets its most important functional and non-functional requirements.
- To ensure that generated solutions are correct and aligned with the context and from so posts.
- To ensure that results are motivated by Stack overflow posts and codebase context.
- To ensure that codebase indexes work.
- To confirm that the tool generates detailed and understable explanations.
- To test that the user can have the facility to choose among modes.
- To test that the user can compare the results between the tool and the llm response.
- To guarantee seamless integration and smooth operation within the VSCode environment.

10.2 Test Cases with execution result

Test cases for the testing of this tool are documented below:

Test Case Id	T1
Title	Installation and Activation
Test Scenario	- Install the StackRAG extension in a fresh VSCode installation. - Activate StackRAG.
Expected Outcome	The extension installs without errors & activates successfully.
Pass/Fail	Pass

Table-16: Test Plan -1

Execution steps:

I opened the vscode extension marketplace. Search the stackrag tool. Then install it. Now it is installed on my computer.

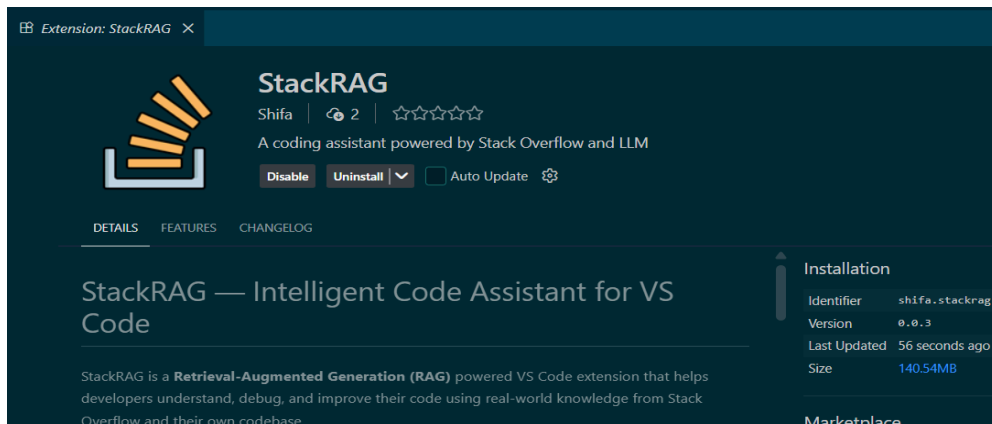


Figure: 40 (successfully installed stackrag extension)

Test Case Id	T2
Title	Chat Interaction
Test Scenario	- Ask a python related question. (e.g., how to write tuple in python) - Check that StackRAG generates output.
Expected Outcome	- StackRAG generates output aligned with the question and Stack Overflow posts.
Pass/fail	- Pass

Table-17: Test Plan - 2

Execution steps:

I asked a question in the chat mode and I got an answer aligned with the question and stack overflow post.



Figure: 41 (receives the output aligned with question and so posts)

Test Case Id	T3
Title	Potential Solutions to Coding Error
Test Scenario	- Request to Fix a coding error (e.g., a recursion code where the base condition is wrong) - Apply the suggested fix provided by StackRAG.
Expected Outcome	The suggested fix accurately resolves the coding error.
pass/fail	- pass

Table-18: Test Plan -3

Execution steps:

I asked to fix a bug in the debug mode and I applied the suggested fix. It correctly resolves the coding error.

fix the bug in the code. the output is not correct after subtracting the list.

Based on the provided information, the bug in the code is due to the incorrect condition in the 'if' statement within the 'subtract_lists' function. The current condition 'if item in y' is causing the function to append items to the result list that are present in both lists 'x' and 'y', which is the opposite of what is intended.

To fix this bug, you need to change the condition to 'if item not in y'. This will ensure that only the elements in 'x' that are not present in 'y' are appended to the result list.

Here is the corrected code:

```
python
def subtract_lists(x, y):
    """
    Returns elements in x that are not in y.
    Expected:
    x = [0,1,2,3,4]
    y = [1,3]
    -> [0,2,4]
    """
    result = []
    for item in x:
        if item not in y: # Changed from 'if item in y' to 'if item not in y'
            result.append(item)
    return result

if __name__ == "__main__":
    x = [0, 1, 2, 3, 4, 5]
    y = [1, 3, 5]
    print(subtract_lists(x, y))
```

Explanation:

Bug Reason: The original code used 'if item in y', which appends items to the result list if they are found in 'y'. This is the opposite of what the function is supposed to do.

Solution: By changing the condition to 'if item not in y', the function will now correctly append items to the result list only if they are not present in 'y'.

Action record:

Ask something...

Debug ☒ Enable Comparison with Copilot ☐ Index ☐ Add File Context ☐ Codebase Summary

Figure: 42 (debug mode result)

Test Case Id	T4
Title	Suggested Refactoring to Code
Test Scenario	- Request to refactor a code with poor structure (e.g., repeatedly performing summation of 10 numbers one by one in separate lines instead of using loop) - Apply the suggested refactoring provided by StackRAG.
Expected Outcome	The suggested refactoring provides better structure while preserving functionality.
pass/fail	pass

Table-19: Test Plan -4

Execution steps:

I asked to refactor a code in the refactor mode and I applied the suggested refactoring. It correctly refactor the code and the code is working.



```
python
from enum import Enum

class BuildType(Enum):
    DEBUG = "debug"
    RELEASE = "release"
    PROFILE = "profile"

def process_build(build_type: BuildType):
    """
    Process build based on type.
    """
    if build_type == BuildType.DEBUG:
        return "Running debug build..."
    elif build_type == BuildType.RELEASE:
        return "Running release build..."
    elif build_type == BuildType.PROFILE:
        return "Running profile build..."
    else:
        raise ValueError("Unknown build type")

if __name__ == "__main__":
    user_input = BuildType.DEBUG
    print(process_build(user_input))

### Changes Made
Ask something...
```

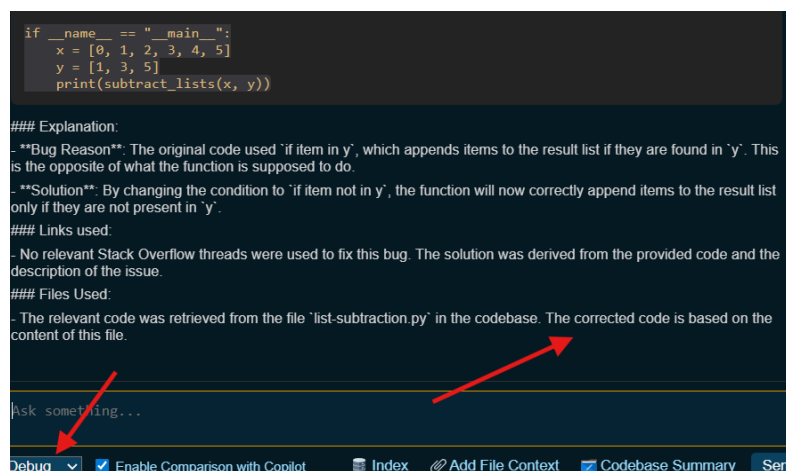
Figure: 43 (refactor mode response)

Test Case Id	T5
Title	Context-Aware Solutions
Test Scenario	- Attach a separate file as context. - Request to fix a bug in the attached code file.
Expected Outcome	StackRAG suggests a potential fix based on the attached context file.
pass/fail	pass

Table-20: Test Plan -5

Execution steps:

I attached a [list-subtraction.py](#) file as a context and the tool provide me bug fixing based on the file content.



```
if __name__ == "__main__":
    x = [0, 1, 2, 3, 4, 5]
    y = [1, 3, 5]
    print(subtract_lists(x, y))

### Explanation:
- **Bug Reason**: The original code used 'if item in y', which appends items to the result list if they are found in 'y'. This is the opposite of what the function is supposed to do.
- **Solution**: By changing the condition to 'if item not in y', the function will now correctly append items to the result list only if they are not present in 'y'.
### Links used:
- No relevant Stack Overflow threads were used to fix this bug. The solution was derived from the provided code and the description of the issue.
### Files Used:
- The relevant code was retrieved from the file 'list-subtraction.py' in the codebase. The corrected code is based on the content of this file.

Ask something...
```

Figure: 44(context aware solution)

Test Case Id	T6
Title	Integration with LLM
Test Scenario	- Ask a question which might not be in the KB - Ensure that the system generates output
Expected Outcome	- StackRAG provides a solution generated by the LLM. -The explanation includes that there are no related posts of SO in the knowledge base. But llm generates the solution based on its own knowledge.
pass/fail	pass

Table-21: Test Plan -6

Execution steps:

I asked to fix a bug. It fixes the bug based on the context and its own knowledge.

```

if __name__ == "__main__":
    x = [0, 1, 2, 3, 4, 5]
    y = [1, 3, 5]
    print(subtract_lists(x, y))

### Explanation:
- Bug Reason**: The original code used 'if item in y', which appends items to the result list if they are found in 'y'. This is the opposite of what the function is supposed to do.
- Solution**: By changing the condition to 'if item not in y', the function will now correctly append items to the result only if they are not present in 'y'.
### Links used:
- No relevant Stack Overflow threads were used to fix this bug. The solution was derived from the provided code and description of the issue.
### Files Used:
- The relevant code was retrieved from the file 'list-subtraction.py' in the codebase. The corrected code is based on the content of this file.

```

Figure: 45 (successfully integrated with LLM)

Test Case Id	T7
Title	KB of Stack Overflow Posts
Test Scenario	- Ask a question on python or javascript language. (e.g., how to write "use strict" in js code)
Expected Outcome	- StackRAG provides a solution with the related SO posts links.
Outcome	- passed

Table-22: Test Plan -7

Execution steps:

I asked a question. The tool provides me with the solution with the link to a related SO post.

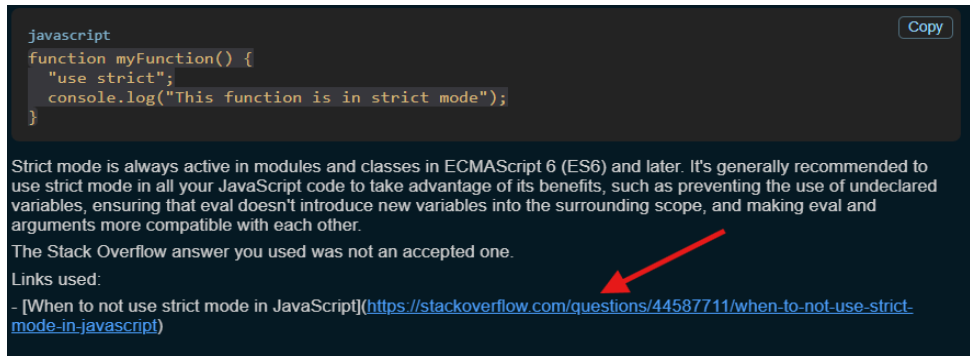


Figure: 46 (utilizes the kB of stack overflow)

Test Case Id	T8
Title	Codebase Indexing
Test Scenario	Index the open codebase. Each code chunk is of two types of categories: function and statement.
Expected Outcome	- generates a codebase indexing for the current open codebase.
pass/fail	pass

Table-23: Test Plan -8

Execution steps:

I generated the indexing of the codebase. The index is correctly generated.

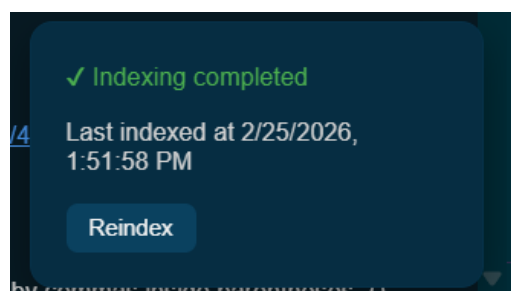


Figure: 47 (successfully indexed codebase)

Test Case Id	T9
Title	Check Codebase Context Extraction Capabilities
Test Scenario	Check that codebase chunks are extracted to generate solutions.
Expected Outcome	- Relevant code context is extracted almost accurately, capturing necessary information for solution generation.
pass/fail	pass

Table-24: Test Plan -9

Execution steps:

After indexing the codebase, for querying in any mode, the tool extracts the related code from the codebase index.

Test Case Id	T10
Title	Side by Side output of stack rag and github copilot.
Test Scenario	- Ask a question related to a python code. - Select the option for side by side output.
Expected Outcome	The system shows us output in the side by side view from where we can select one output to continue.
pass/fail	pass

Table-25: Test Plan -10

Execution steps:

If the compare mode is enabled, then it generates the output side by side view along with the dif showing. And we can select one output to continue with it.

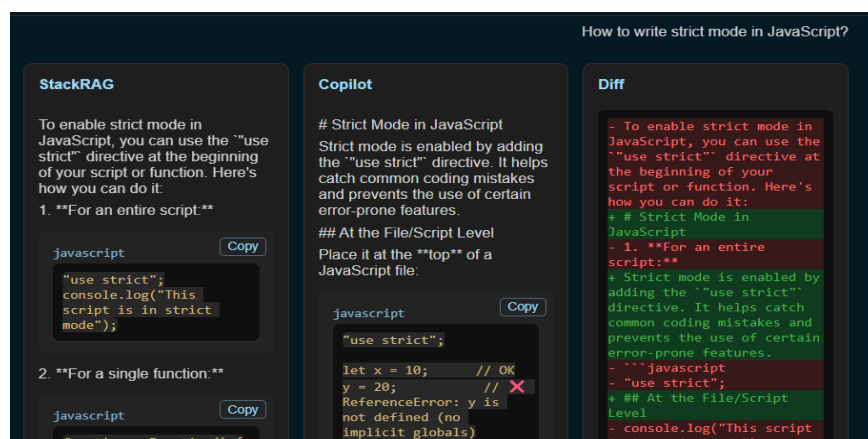


Figure: 48 (Side by Side output of stack rag and github copilot)

Test Case Id	T11
Title	Codebase summary
Test Scenario	- Ask to generate a summary for the currently open codebase.
Expected Outcome	The system gives us a summary of the currently open codebase by using the codebase indexing.
pass/fail	pass

Table-26: Test Plan -11

Execution steps:

After prompting for codebase summary, the tool generates the codebase summary.

```

Generate a summary of the current codebase.

### Project Overview
- A collection of utility functions and scripts written in JavaScript and Python.
- Focuses on basic data manipulation, string operations, and build processing.
- Contains both active and commented-out code (likely for testing or historical reference).
### Tech Stack
- **Languages**: JavaScript, Python
- **No external libraries/frameworks** used (pure vanilla implementations).
### Architecture
- **Modular**: Each file contains a single, focused utility function.
- **No clear project structure**: Files appear independent (no shared dependencies or common patterns).
- **Testing**: Some files include test cases (e.g., 'console.log' or 'if __name__ == "__main__":' blocks).
### Key Components
1. **range_generator.js**
- Generates an array of sequential numbers (1 to 'n').
- Includes two implementations (one active, one commented out).
2. **remove_duplicate.js**
- Filters duplicate names from an array.
- Contains a corrected version (fixed '===' -> '===' logic).
3. **list_subtraction.py**
- Returns elements in list 'x' not present in list 'y'.
- Includes a corrected version (fixed 'not in' logic).
4. **currency_formatter.js**
- Formats a number as a currency string (e.g., '$ 2500.00').
5. **build_processor.py**
- Processes build types ('debug', 'release', 'profile') with conditional logic.

```

Figure: 49 (codebase summary)

Test Case Id	T12
Title	Reindexing the codebase
Test Scenario	- Reindex the codebase.
Expected Outcome	StackRAG refreshes the stale indexing.
pass/fail	pass

Table-27: Test Plan -12

Execution steps:

I clicked on the reindex option and it successfully reindexed the codebase.

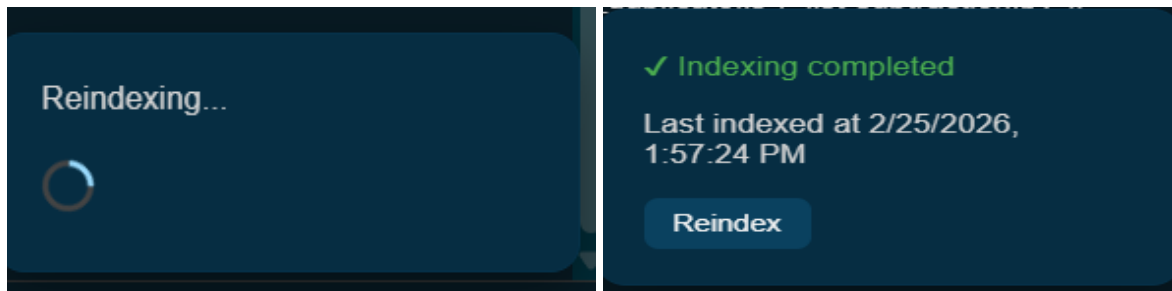


Figure: 50 (codebase reindexing)

Conclusion

In the current report, I have included all the detailed documentation of the **StackRAG tool**: A LLM and RAG powered AI Coding Assistant AS VS Code Extension. I have elaborated on the project scope, requirements, and usage scenarios in detail. Scenario-based modeling, data modeling and class-based modeling were done to establish a resilient system architecture. After that i have included the system design, design classes, implementation details, user manual and deployment diagram. In addition, a general overview of the goals of testing as well as a summary of the features to be tested, the test results have been presented.

While StackRAG can handle chat interaction, bug fixing, code refactoring, It might be further improved in many ways: extending it to work with different IDEs and continuing to use more advanced RAG techniques to improve the solution based on context.

Reference

- [1] Davit Abrahamyan, Fatemeh H. Fard. "StackRAG Agent: Improving Developer Answers with Retrieval-Augmented Generation." 40th International Conference on Software Maintenance and Evolution (ICSME), 2024. [Link](#)
- [2] Elijah Mansur, Johnson Chen, Muhammad Anas Raza, Mohammad Wardat. "RAGFix: Enhancing LLM Code Repair Using RAG and Stack Overflow Posts." 2024 IEEE International Conference on Big Data (BigData). [Link](#)
- [3] Tasmia Zerin, Moumita Asad, B M Mainul Hossain, Kazi Sakib. "Repairing Responsive Layout Failures Using Retrieval Augmented Generation" in Proceedings of the 41st International Conference on Software Maintenance and Evolution (ICSME), 2025. [Link](#)
- [4] [google survey form](#)